Dev Ops Project-2 Assignment of TrendStore Application Deployment

Deploy the given React application to a production ready state

Prepared by: Md Rustam Ansari *Hcl Gui Batch:* DO-C-WE-E-B36

- *Project Scope URL & Refer the Guidelines:* https://docs.google.com/document/d/114RStvA92SmiU2uzsvMW-4LR6Yjq13mKCyXYyIvaBOM

Project Tasks contents & Timeline

SN.	Task Descriptions	Page	Duration
Phase 1:	Project Scope Summary and Guidelines	1-1	-- --
Phase 2:	Install Git bash using Winget	2-2	-- --
Phase 3:	AWS EC2 Infrastructure & Security Group	3-3	-- --
Phase 4:	Version Control (GitHub, Git Clone)	3-4	-- --
Phase 5:	Dockerization (Docker Hub & Git CLI)	4-6	-- --
Phase 6:	Bash Scripting (Write 2 Bash Script)	6-6	-- --
Phase 7:	Jenkins Configuration & Auto-Trigger Setup	7-7	-- --
Phase 8:	Open-Source Health Monitoring & Alerts	8-7	-- --
Phase 9:	Final Verification & Submission Checklist	6-6	-- --

Tech stacks need to be used:

- AWS
- Docker
- DevOps
- Version Control

How do I submit my work?

- Push all your work files to GitHub (O/P screenshot images must).
- Submit your project to the version control public repository to verify them.

Terms and Conditions?

- Submit your project to the version control public repository to verify them.
- You agree to not share this confidential document with anyone.
- You agree to open-source your code (it may even look good on your profile!).
- Follow the following workflow to perform assigned project accordingly.
- Do not mention our company's name anywhere in the code. We will never use your source code under any circumstances for any commercial purposes; this is just a basic assessment task.

NOTE: Any violation of Terms and conditions is strictly prohibited. You are bound to adhere to it.

○ *Phase 2: AWS Command Line Interface (AWS CLI)*

- *Download and install the AWS CLI on Windows laptop*
- *Configure and get access the AWS using the AWS CLI*

○ *Workflow Phase 2 (Install AWS CLI, Configure them):*

- *Open CMD and type bash to open command line interface (insure wsl, bash already installed)*
- *Validate the AWS cli installed on system*

$ aws --version

```
C:\Users\Md Rustam>bash
md_rustam@DESKTOP-CPK0PUB:/mnt/c/Users/Md Rustam$ aws --version
aws-cli/2.37.4 Python/3.14.6 Linux/6.6.114.1-microsoft-standard-WSL2 script-exe/x86_64.ubuntu.26
```

- Ensure already configured the AWS CLI

C:\Users\Md Rustam>aws configure list

```
C:\Users\Md Rustam>aws configure list
NAME        : VALUE                    : TYPE             : LOCATION
profile     : <not set>                : None             : None
access_key  : ****************OMVR     : shared-credentials-file :
secret_key  : ****************hhBT     : shared-credentials-file :
region      : us-east-1                : config-file      : ~/.aws/config
```

- Validate the AWS CLI is accessibled

$ aws configure get region

```
md_rustam@DESKTOP-CPK0PUB:/mnt/c/Users/Md Rustam$ aws configure get region
ap-southeast-2
```

○ *Phase 3: AWS EC2 Infrastructure & Security Group:*

- *Launch EC2 Instance as the project client instance*

○ *Workflow Phase 3 (Launch AWS Instance for access as Client):*

- Created a simple bash script to Launch ubuntu 26.04 aws ec2 instance with the default settings

$ bash aws-ec2-instance.sh

```
md_rustam@DESKTOP-CPK0PUB:~$ bash aws-ec2-instance.sh
1. Authorizing SSH (103.95.166.8/32) on Default Security Group...
{
    "Return": true,
    "SecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-0eb26a24a9cdfe200",
            "GroupId": "sg-0c7439b8f5653a0de",
            "GroupOwnerId": "633697460172",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 22,
            "ToPort": 22,
            "CidrIpv4": "103.95.166.8/32",
            "SecurityGroupRuleArn": "arn:aws:ec2:ap-southeast-2:633697460172:security-group-rule/sgr-0eb26a24a9cdfe200"
        }
    ]
}
2. Setting up SSH Key (keypair.pem)...
3. Launching Ubuntu 26.04 EC2 Instance...
4. Waiting for instance (i-03289d637d233c858) to start...
5. Waiting for SSH on 54.66.151.38...
Connected! (To reconnect later: ssh -i keypair.pem ubuntu@54.66.151.38)
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-1014-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Thu Oct 22 05:19:08 UTC 2026

  System load:    1.22      Processes:             23
  Usage of /home: unknown   Users logged in:       0
  Memory usage:   3%        IPv4 address for eth0: 10.10.10.2
  Swap usage:     0%


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
```

- *Update the latest OS patch, install prerequisite packages and AWS CLI in the newly created instance*

$ sudo apt update -y; sudo apt upgrade -y; sudo apt install awscli -y

```
ubuntu@ip-172-31-33-199:~$ sudo apt update -y; sudo apt upgrade -y; sudo apt install awscli -y
Hit:1 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
All packages are up to date.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
awscli is already the newest version (2.31.35-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

- *The default security group typically does not allow inbound SSH (port 22), allow them.*

$ aws ec2 describe-instances --region ap-southeast-2 --query 'Reservations[*].Instances[*].[InstanceId,State.Name]' --output table

```
ubuntu@ip-172-31-33-199:~$ aws ec2 describe-instances   --region ap-southeast-2   --query 'Reservations[*].Instances[*].[InstanceId,State.Name]'   --output table
-------------------------------------
|        DescribeInstances          |
+---------------------+-------------+
|  i-03289d637d233c858 |  running   |
+---------------------+-------------+
```

○ *Phase 4: Version Control (GitHub, Git Clone)*

- *Clone the below mentioned repository and deploy the application (Run application in port 3000).*
- *Git Repo Path: https://github.com/Vennilavanguvi/Trend.git*

○ *Workflow Phase 4 (Create a New Repository on GitHub)*

- *Clone the below mentioned repository under directory Final_Project-2*
- *Navigate the Project-2_React path and check the .gitignore and .dockerignore files*

- *Create a directory "Final_Project-2" into the ec2 instance and make git clone.*

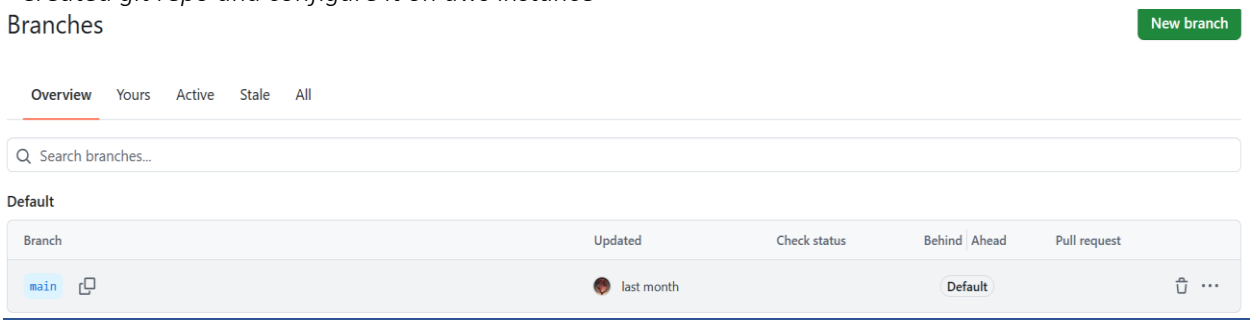
$ git clone https://github.com/Vennilavanguvi/Trend.git

```
ubuntu@ip-172-31-33-199:~/Final_Project-2$ git clone https://github.com/Vennilavanguvi/Trend.git
Cloning into 'Trend'...
remote: Enumerating objects: 80, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 80 (delta 0), reused 0 (delta 0), pack-reused 77 (from 1)
Receiving objects: 100% (80/80), 8.58 MiB | 23.50 MiB/s, done.
Resolving deltas: 100% (1/1), done.
```

- *Navigate to the cloned Git repository directory and list the contents of the directory and start the serve*

```
ubuntu@ip-172-31-33-199:~/Final_Project-2$ ls -l Trend/
total 8
-rw-rw-r-- 1 ubuntu ubuntu 1710 Oct 26 18:23 README.md
drwxrwxr-x 3 ubuntu ubuntu 4096 Oct 26 18:23 dist
```

- *Created git repo and configure it on aws instance*



- *Create .gitignore and .dockerignore*

$ cd /Final_Project-2/Trend

```
cat <<EOF > .gitignore
node_modules
build
.terraform/
*.tfstate*
*.pem
EOF

cat <<EOF > .dockerignore
node_modules
.git
.gitignore
Dockerfile
README.md
EOF
```

○ *Phase 5: Docker_Containarization Application:*

- *Dockerize the application by creating Dockerfile*
- *Build an application and check output using docker image.*
-

○ *Workflow Phase 5 (Docker_Containarization):*

- *Dockerize the Application & Test Locally post deploy the docker required packages*
- *Create a dockerfile to build and serve the React application on port 3000, then verify it locally.*

$ sudo apt update && sudo apt install -y docker.io docker-compose-v2 git; sudo usermod -aG docker $USER; newgrp docker

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ sudo apt update -y; sudo apt upgrade -y; sudo apt install awscli -y
Hit:1 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease
All packages are up to date.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
awscli is already the newest version (2.31.35-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

- *Create a DockerHub Repository: Log in to DockerHub and create a public repository named*

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ docker login

USING WEB-BASED LOGIN

i Info → To sign in with credentials on the command line, use 'docker login -u <username>'


Your one-time device confirmation code is: TCML-ZSCP
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser…

WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

- *Create a Dockerfile to build the TrendStore files and serve them on port 3000 via Nginx*

$ cat dockerfile

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ cat > dockerfile << EOF
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
RUN npm install -g serve
EXPOSE 3000
CMD ["serve", "-s", "build", "-l", "3000"]
EOF
```

- *Build and test the docker image $ cat docker-compose.yml*

$ docker build-t trend-app:v1 .

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ docker build -t trend-app:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon   9.29MB
Step 1/4 : FROM nginx:alpine
alpine: Pulling from library/nginx
7cc330898115: Pulling fs layer
b9b388a21ecf: Pulling fs layer
26185d08a3ec: Pulling fs layer
2ec5fb117ffa: Pulling fs layer
9b0f7dfe35b2: Pulling fs layer
12b85cff06fd: Pulling fs layer
7b229517ed22: Pulling fs layer
657023555a2a: Pulling fs layer
7cc330898115: Download complete
26185d08a3ec: Download complete
12b85cff06fd: Download complete
7b229517ed22: Download complete
657023555a2a: Download complete
2ec5fb117ffa: Download complete
9b0f7dfe35b2: Download complete
b9b388a21ecf: Download complete
7b229517ed22: Pull complete
a4d5b1f57abc: Download complete
acc2d54c3743: Download complete
7cc330898115: Pull complete
26185d08a3ec: Pull complete
12b85cff06fd: Pull complete
657023555a2a: Pull complete
2ec5fb117ffa: Pull complete
9b0f7dfe35b2: Pull complete
b9b388a21ecf: Pull complete
Digest: sha256:a3554c0d0b48177b1413c26e01f695334428020d9df01243fbae3e11cfd17417
Status: Downloaded newer image for nginx:alpine
 ---> a3554c0d0b48
Step 2/4 : COPY dist/ /usr/share/nginx/html/
 ---> 38836af70734
Step 3/4 : EXPOSE 80
 ---> Running in 51919af74da9
 ---> Removed intermediate container 51919af74da9
 ---> ecc263c7da21
Step 4/4 : CMD ["nginx","-g","daemon off;"]
 ---> Running in 6579e2ece074
 ---> Removed intermediate container 6579e2ece074
 ---> 3cfeae2f7889
Successfully built 3cfeae2f7889
Successfully tagged trend-app:v1
```

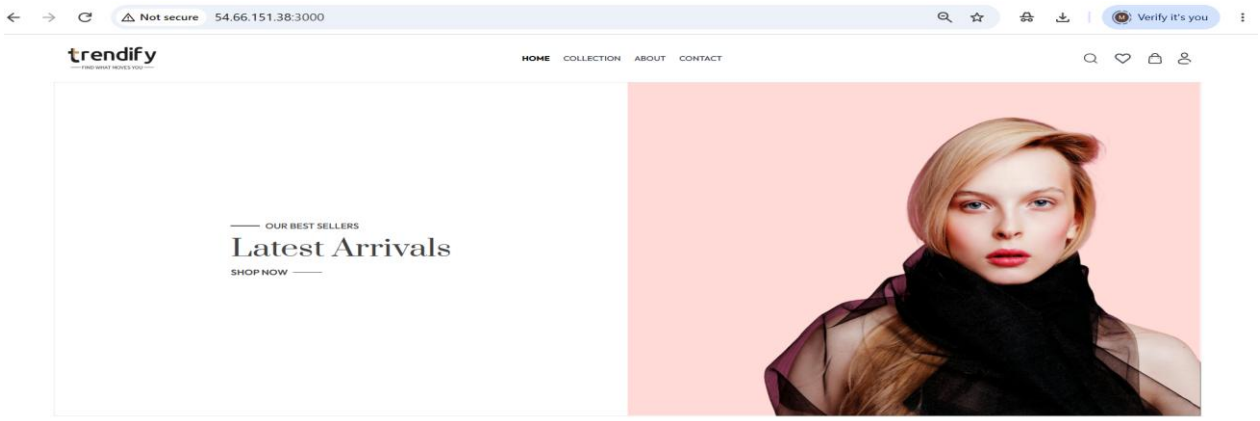
```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ docker images

                                                                    i Info →  U  In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
doch2/trend-dev:v1   3cfeae2f7889        112MB         35.3MB    U
trend-app:v1         3cfeae2f7889        112MB         35.3MB    U
```

$ docker run-d-p 3000:80--name trend-test trend-app:v1

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ docker run -d \
--name trend-test \
-p 3000:80 \
trend-app:v1
bccdee9331f4a10b75d8e0516e4fcd7768dc22dbf0930eaeeea07923a7a91e73
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                                       NAMES
bccdee9331f4   trend-app:v1   "/docker-entrypoint.…"   7 seconds ago   Up 6 seconds   0.0.0.0:3000->80/tcp, [::]:3000->80/tcp   trend-test
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ curl localhost:3000
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Trendify | Find What Moves You</title>
    <script type="module" crossorigin src="/assets/index-BjbOuBLy.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-B52p61-W.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

Verify at http://localhost:3000, then remove the test container:
docker rm-f trend-test



$ docker tag trend-app:v1 doch2/trend-dev:v1

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ docker tag trend-app:v1 doch2/trend-dev:v1
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ docker push doch2/trend-dev:v1
The push refers to repository [docker.io/doch2/trend-dev]
657023555a2a: Mounted from library/nginx
26185d08a3ec: Mounted from library/nginx
2fc7a016ff49: Pushed
7b229517ed22: Mounted from library/nginx
b9b388a21ecf: Mounted from library/nginx
2ec5fb117ffa: Mounted from library/nginx
9b0f7dfe35b2: Mounted from library/nginx
7cc330898115: Mounted from library/nginx
12b85cff06fd: Mounted from library/nginx
v1: digest: sha256:3cfeae2f788957b0c5ade5662e8d406e6bdf26c5da067ff39b19b8e7b09ac7bf size: 2265
```

- Write Kubernetes Manifests: Create the deployment and service YAML files so the application runs on AWS EKS and exposes a LoadBalancer on port 3000

$ cat deployment.yaml

```
ubuntu@ip-172-31-33-199:~/Final_Project-2$ cat deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: trend-deployment
  labels:
    app: trend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: trend
  template:
    metadata:
      labels:
        app: trend
    spec:
      containers:
      - name: trend-app
        image: <YOUR_DOCKERHUB_USERNAME>/trend-app:latest
        ports:
        - containerPort: 3000
```

$ cat service.yaml

```
ubuntu@ip-172-31-33-199:~/Final_Project-2$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: trend-service
spec:
  type: LoadBalancer
  selector:
    app: trend
  ports:
    - protocol: TCP
      port: 3000
      targetPort: 3000
```

○ *Phase 6: Terraform Infrastructure (Create Infrastructure)*

- Define infrastructure in main.tf to create VPC, IAM, EC2 with Jenkins, etc.

- Use terraform command to provision infrastructure.

○ *Workflow Phase 6 (Installation and configure Jenkins build).*

- Run the terraform commands to initialize

- $ wget -O- https://apt.releases.hashicorp.com/gpg | \ gpg --dearmor | \ sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg > /dev/null

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
sudo tee /etc/apt/sources.list.d/hashicorp.list
deb [arch=amd64 signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com resolute main
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ sudo apt-get update
sudo apt-get install -y terraform
Hit:1 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:3 http://ap-southeast-2.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:4 https://apt.releases.hashicorp.com resolute InRelease [12.9 kB]
Get:5 https://apt.releases.hashicorp.com resolute/main amd64 Packages [332 kB]
Hit:6 http://security.ubuntu.com/ubuntu resolute-security InRelease
Fetched 345 kB in 1s (548 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following NEW packages will be installed:
  terraform
```

$ terraform -version; terraform init; terraform validate

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Trend$ terraform -version; terraform init; terraform validate
Terraform v1.17.0
on linux_amd64

Terraform initialized in an empty directory!

The directory has no Terraform configuration files. You may begin working
with Terraform immediately by creating Terraform configuration files.
Success! The configuration is valid.
```

- Create EKS Cluster

$ eksctl create cluster --name trend-cluster --region ap-southeast-2 --nodes 2

- Configure kubectl:

$ aws eks update-kubeconfig --region ap-southeast-2 --name trend-cluster

- Verify: kubectl get nodes

- Deploy:

$ kubectl apply -f deployment.yaml

$ kubectl apply -f service.yaml

- Check:

$ kubectl get pods

$ kubectl get svc

- GitHub Webhook: Settings → Webhooks → Add Webhook

○ *Phase 7: Create the Declarative Jenkins Pipeline*

- Install Jenkins and necessary plugins (Docker, Git, Kubernetes, Pipeline) for build, push & deploy applications.

- Setup Github and jenkins integration using github webhook build trigger for auto build for every commit.

- *Create a declarative pipeline script and pipeline project to build, push & deploy using CI-CD.*

○ *Workflow Phase 7 (Jenkins Pipeline / CICD Pipeline).*

- Create the Declarative Jenkins Pipeline: Create a Jenkinsfile in the root directory to automate building the image, pushing to DockerHub, and deploying to EKS via kubectl

$ cat Jenkinsfile

```
ubuntu@ip-172-31-33-199:~/Final_Project-2$ cat Jenkinsfile
pipeline {
    agent any
    environment {
        DOCKERHUB_CREDENTIALS = credentials('dockerhub-creds')
        IMAGE_NAME = "<YOUR_DOCKERHUB_USERNAME>/trend-app"
        AWS_REGION = "ap-south-1"
        EKS_CLUSTER_NAME = "trend-eks-cluster"
    }
    stages {
        stage('Checkout SCM') {
            steps {
                git branch: 'main', url: 'https://github.com/<YOUR_GITHUB_USERNAME>/Trend.git'
            }
        }
        stage('Build Docker Image') {
            steps {
                sh 'docker build -t ${IMAGE_NAME}:${BUILD_NUMBER} -t ${IMAGE_NAME}:latest .'
            }
        }
        stage('Push to DockerHub') {
            steps {
                sh 'echo $DOCKERHUB_CREDENTIALS_PSW | docker login -u $DOCKERHUB_CREDENTIALS_USR --password-stdin'
                sh 'docker push ${IMAGE_NAME}:${BUILD_NUMBER}'
                sh 'docker push ${IMAGE_NAME}:latest'
            }
        }
        stage('Deploy to EKS') {
            steps {
                withCredentials([[$class: 'AmazonWebServicesCredentialsBinding', credentialsId: 'aws-creds']]) {
                    sh '''
                        aws eks update-kubeconfig --region ${AWS_REGION} --name ${EKS_CLUSTER_NAME}
                        kubectl apply -f deployment.yaml
                        kubectl apply -f service.yaml
                        kubectl rollout restart deployment/trend-deployment
                    '''
                }
            }
        }
    }
}
```

○ Run Jenkins

$ docker volume create jenkins_home

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Final_Project-2/Trend$ docker volume create jenkins_home
jenkins_home
ubuntu@ip-172-31-33-199:~/Final_Project-2/Final_Project-2/Trend$ docker run -d \
--name jenkins \
-p 8080:8080 \
-p 50000:50000 \
-v jenkins_home:/var/jenkins_home \
-v /var/run/docker.sock:/var/run/docker.sock \
jenkins/jenkins:lts-jdk21
Unable to find image 'jenkins/jenkins:lts-jdk21' locally
lts-jdk21: Pulling from jenkins/jenkins
a91427fe0e3f: Pull complete
810008851f3a: Pull complete
f9e31fab50e8: Pull complete
95bc7464aaea: Pull complete
d89089c8a5a3: Pull complete
968c36a3b2c5: Pull complete
701ded2f9b33: Pull complete
6eefb2f5d3e9: Pull complete
408777b27aa9: Pull complete
3dd4a99993ab: Pull complete
a20a14499a3e: Pull complete
463a574793f8: Pull complete
44136fa355b3: Download complete
9613b4f6b0c7: Download complete
Digest: sha256:a660310e39ade10631f774bacd5219de767dfd08947ea5c32a739b5e5bb382c1
Status: Downloaded newer image for jenkins/jenkins:lts-jdk21
da13bb8d7b35e20336fc87ccf27c21bcc2c52359fd46127aac082e70dce3d84b
```

○ Get Password:

$ docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword

```
ubuntu@ip-172-31-33-199:~/Final_Project-2/Final_Project-2/Trend$ docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
03fdbd27b7014be8b5ac63b4fc2d8405
```

- *Phase 8: Setup Open-Source Monitoring (Prometheus & Grafana)*

- *Setup a monitoring system to check the health of the cluster or application (opensource) is highly appreciable.*

- *Workflow Phase 7 (Open-Source Health Monitoring & Alerts)*

- *Open-Source Monitoring & Alerting Setup*
- *Deploy the kube-prometheus-stack using Helm on your EKS cluster to monitor cluster and application health.*

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-chart
helm repo update
kubectl create namespace monitoring
helm install monitoring prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set grafana.service.type=LoadBalancer
```

- *Run Uptime Kuma on your EC2 instance:*

```
docker run -d \
  --restart=always \
  -p 3001:3001 \
  -v uptime-kuma:/app/data \
  --name uptime-kuma \
  louislam/uptime-kuma:1
```

- *Open http://54.66.151.38:3001 and create an admin account.*
- *Click Add New Monitor:*
  - *Monitor Type: HTTP(s)*
  - *Friendly Name: React Production App*
  - *URL: http://54.66.151.38:80*
  - *Heartbeat Interval: 60 seconds*

- *Click Setup Notification (on the right panel):*
  - *Select your preferred provider (e.g., Email (SMTP), Slack, Telegram, or Discord).*
  - *Configure the webhook/SMTP details, test the alert, and click Save.*
  - *Uptime Kuma will now trigger notifications only when the container on port 80 stops responding.*

- *Phase 8: Final Verification & Submission Checklist*
- *Workflow Phase 7 (Final Verification & Submission Checklist)*

- *Create a screenshots/ folder or update README.md in your GitHub repository with the required submission details and screenshots.*
- *git add .; git commit -m "19_Monitoring_Task_2"; git push origin main*

Thanks